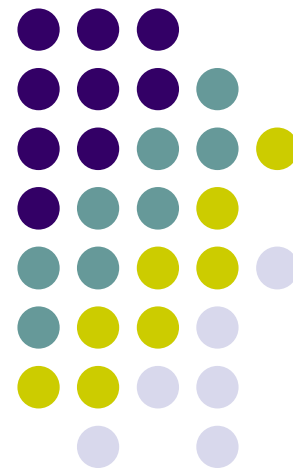
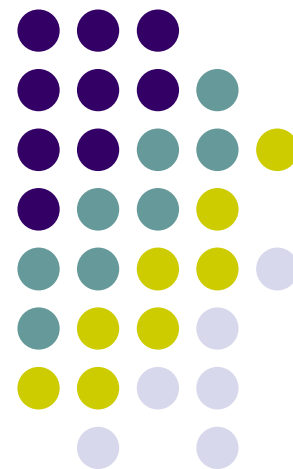


第2章

递归与分治策略



递归篇



上节课提到的插入排序

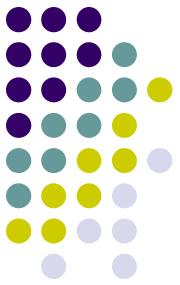


算法效率分析

约定：在算法伪代码中，执行一行代码仅花常数时间。第*i*行花费常数时间 c_i

		Cost	times
InsertSort(A)			
1	for $j \leftarrow 2$ to $length[A]$	C_1	n
2	do $key \leftarrow A[j]$	C_2	$n-1$
3	//Insert $A[j]$ into the sorted $A[1..j-1]$	0	$n-1$
4	$i \leftarrow j - 1$	C_4	$n-1$
5	while $i > 0$ and $A[i] > key$ do	C_5	$\sum_{j=2}^n t_j$
6	$A[i + 1] \leftarrow A[i]$	C_6	$\sum_{j=2}^n (t_j - 1)$
7	$i \leftarrow i - 1$	C_7	$\sum_{j=2}^n (t_j - 1)$
8	$A[i + 1] \leftarrow key$	C_8	$n-1$

t_j 表示在for循环的迭代j中，while循环的次数。



- 把算法执行每行的次数与执行每行所需要的时间相乘，然后把所有的时间求和：

$$T(n) = c_1 n + c_2 (n-1) + c_4 (n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) \\ + c_7 \sum_{j=2}^n (t_j - 1) + c_8 (n-1)$$



```
5   while  $i > 0$  and  $A[i] > \text{key}$  do
6        $A[i + 1] \leftarrow A[i]$ 
7        $i \leftarrow i - 1$ 
```

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

➤ 如果数组是排好序的，则会出现最好情况：

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) = an + b \end{aligned}$$

➤ 如果数组是逆序排序的，则会出现最差情况：

此时必须将每个元素 $A[j]$ 与整个已排序的子数组 $A[1..j-1]$ 中的每一个元素进行比较，对 $j=2,3,\dots,n$ ，有 $t_j=j$ 。则有：

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1, \quad \sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

总时间：

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) \\ &\quad + c_6\left(\frac{n(n-1)}{2}\right) + c_7\left(\frac{n(n-1)}{2}\right) + c_8(n-1) = an^2 + bn + c \end{aligned}$$



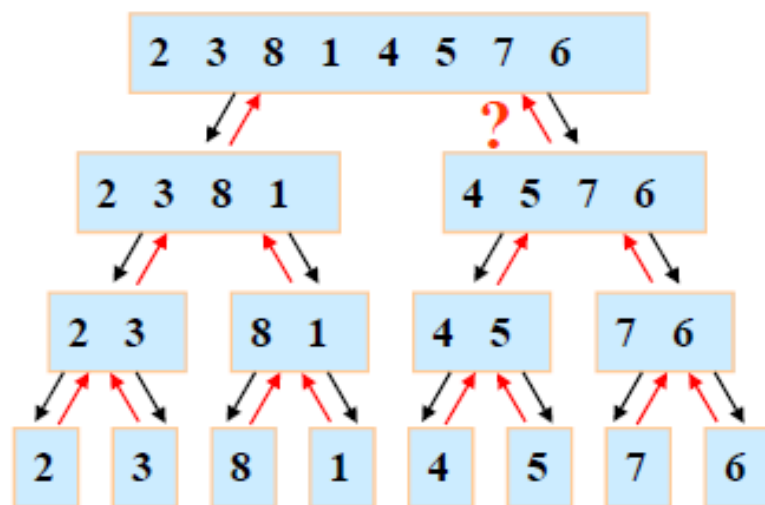
- 对于一个问题，可以有很多中解决方法。因此，算法设计策略也有很多。
- 排序问题：
 - ◆ Bubble sort: bubbling
 - ◆ Insertion sort: incremental approach (增量靠近)
 - ◆ Merge sort: divide-and conquer (分而治之)
 - ◆ Quick sort: location (元素定位)
 -
- 分治法最差的时间比插入排序法差得多



- 归并排序算法 (Merge sort algorithm)

- ① 分解: 把 n 个元素分成各含 $n/2$ 个元素的子序列;
- ② 解决: 用归并排序算法对两个子序列递归地排序;
- ③ 合并: 合并两个已排序的子序列以得到排序结果。

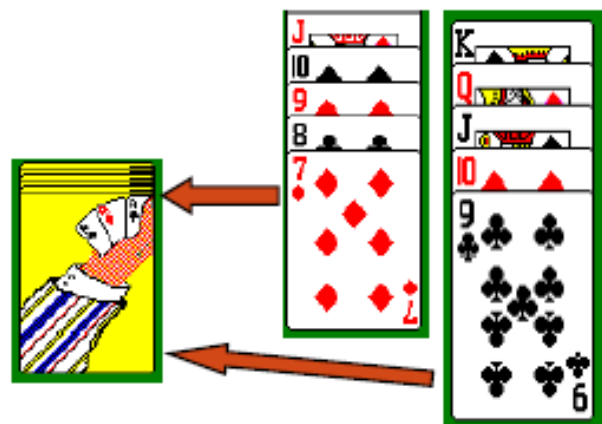
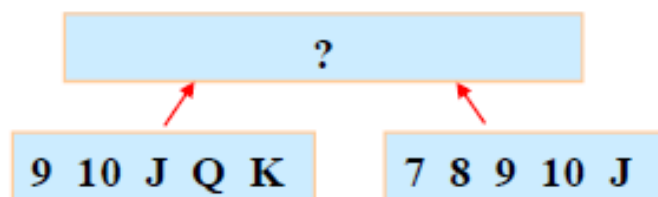
在对子序列排序时, 其长度为1时递归结束。
单个元素被视为是已排好序的。





- $\text{MERGE}(A, p, q, r)$: 归并排序算法的关键步骤，合并步骤中合并两个已经排序好的子序列。

- ◆ A 是个数组， p, q, r 数组中元素的下标，且 $p \leq q < r$.
- ◆ 该过程假设子数组 $A[p \dots q]$ 和 $A[q+1 \dots r]$ 是有序的，并将它们合并成一个已排好序的子数组代替当前子数组 $A[p \dots r]$ 。
- ◆ 合并过程:



The procedure takes time $\Theta(n)$, $n=r-p+1$ 。



MERGE(A, p, q, r)

```
1   $n_1 \leftarrow q-p+1$ 
2   $n_2 \leftarrow r-q$ 
3  create arrays  $L[1 .. n_1+1]$  and  $R[1 .. n_2+1]$ 
4  for  $i \leftarrow 1$  to  $n_1$ 
5      do  $L[i] \leftarrow A[p+i-1]$ 
6  for  $j \leftarrow 1$  to  $n_2$ 
7      do  $R[j] \leftarrow A[q+j]$ 
8   $L[n_1+1] \leftarrow \infty$     // To avoid having to check whether either pile is empty in each
9   $R[n_2+1] \leftarrow \infty$     // basic step, a sentinel card is put on the bottom of each pile.
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13     do if  $L[i] \leq R[j]$ 
14         then  $A[k] \leftarrow L[i]$ 
15              $i \leftarrow i+1$ 
16         else  $A[k] \leftarrow R[j]$ 
17              $j \leftarrow j+1$ 
```

How does the subroutine work? Next Page

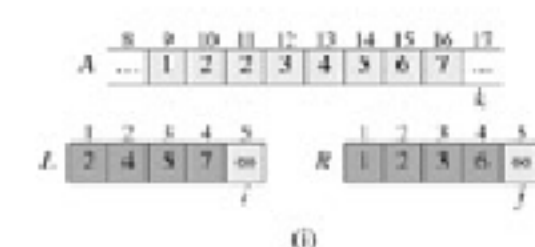
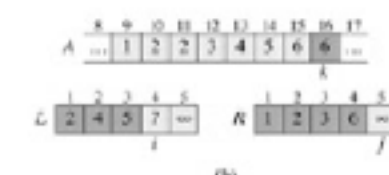
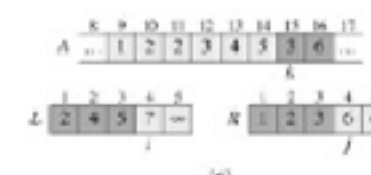
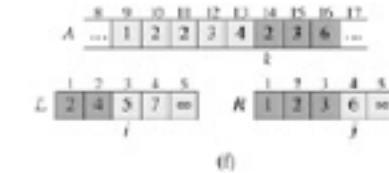
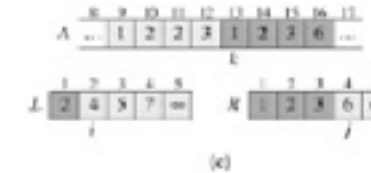
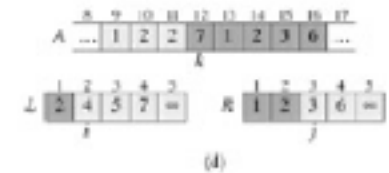
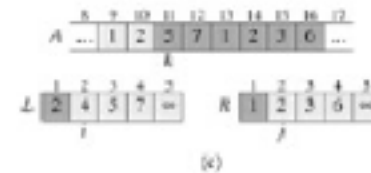
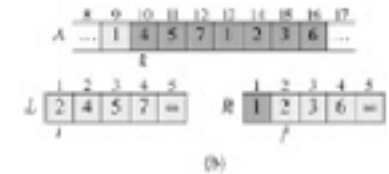
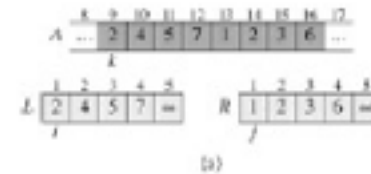


MERGE(A, p, q, r)

```

1   $n_1 \leftarrow q - p + 1$ 
2   $n_2 \leftarrow r - q$ 
3  create arrays  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$ 
4  for  $i \leftarrow 1$  to  $n_1$ 
5      do  $L[i] \leftarrow A[p + i - 1]$ 
6  for  $j \leftarrow 1$  to  $n_2$ 
7      do  $R[j] \leftarrow A[q + j]$ 
8   $L[n_1 + 1] \leftarrow \infty$ 
9   $R[n_2 + 1] \leftarrow \infty$ 
10  $i \leftarrow 1$ 
11  $j \leftarrow 1$ 
12 for  $k \leftarrow p$  to  $r$ 
13     do if  $L[i] \leq R[j]$ 
14         then  $A[k] \leftarrow L[i]$ 
15              $i \leftarrow i + 1$ 
16         else  $A[k] \leftarrow R[j]$ 
17              $j \leftarrow j + 1$ 

```





MERGE(A, p, q, r)	cost	times
1 $n_1 \leftarrow q-p+1$	c	1
2 $n_2 \leftarrow r-q$	c	1
3 create arrays $L[1 \dots n_1+1]$ and $R[1 \dots n_2+1]$	c	1
4 for $i \leftarrow 1$ to n_1	c	n_1+1
5 do $L[i] \leftarrow A[p+i-1]$	c	n_1
6 for $j \leftarrow 1$ to n_2	c	n_2+1
7 do $R[j] \leftarrow A[q+j]$	c	n_2
8 $L[n_1+1] \leftarrow \infty$	c	1
9 $R[n_2+1] \leftarrow \infty$	c	1
10 $i \leftarrow 1$	c	1
11 $j \leftarrow 1$	c	1
12 for $k \leftarrow p$ to r	c	$r-p+2$
13 do if $L[i] \leq R[j]$	c	$r-p+1$
14 then $A[k] \leftarrow L[i]$	c	x
15 $i \leftarrow i+1$	c	x
16 else $A[k] \leftarrow R[j]$	c	$r-p+1-x$
17 $j \leftarrow j+1$	c	$r-p+1-x$

$$r-p+1 \\ = n_1 + n_2 = n$$

$$1 \leq x \leq n_1$$

$$\Theta(n_1+n_2)=\Theta(n)$$



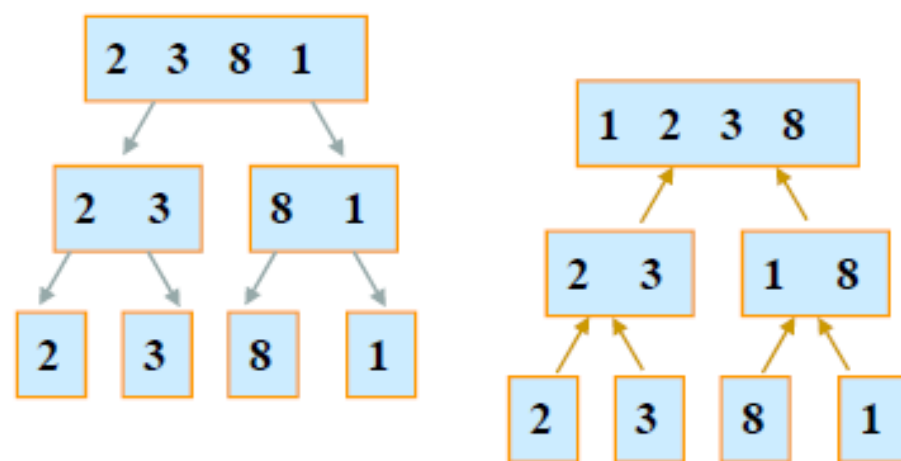
- 将MERGE过程作为合并排序中的一个子过程来使用。下面过程MERGE-SORT(A, p, r)对子数组 $A[p..r]$ 进行排序。如果 $p \geq r$, 则该子数组中至多只有一个元素, 当然就是已排序的。否则, 分解步骤就计算出一个下标 q , 将 $A[p..r]$ 分成 $A[p..q]$ 和 $A[q+1..r]$, 各含 $\lceil n/2 \rceil$ 个元素。

```
MERGE-SORT( $A, p, r$ )  
1  if  $p < r$   
2      Then  $q \leftarrow \lfloor (p+r)/2 \rfloor$   
3          MERGE-SORT( $A, p, q$ )  
4          MERGE-SORT( $A, q+1, r$ )  
5          MERGE( $A, p, q, r$ )
```

MERGE-SORT(A, p, r)

```

1  if  $p < r$ 
2    Then  $q \leftarrow$ 
3      MERGE-SORT( $A, p, q$ )
4      MERGE-SORT( $A, q+1, r$ )
5      MERGE( $A, p, q, r$ )
  
```



MERGE-SORT($A, 1, 4$)

```

1  if  $1 < 4$ 
2    Then  $q \leftarrow \lfloor (1+4)/2 \rfloor = 2$ 
3    MERGE-SORT( $A, 1, 2$ )
      1  if  $1 < 2$ 
      2  Then  $q \leftarrow \lfloor (1+2)/2 \rfloor = 1$ 
      3  MERGE-SORT( $A, 1, 1$ )
          1  if  $1 < 1$ 
      4  MERGE-SORT( $A, 2, 2$ )
          1  if  $2 < 2$ 
      5  MERGE( $A, 1, 1, 2$ )
4  MERGE-SORT( $A, 3, 4$ )
      1  if  $3 < 4$ 
      2  Then  $q \leftarrow \lfloor (3+4)/2 \rfloor = 3$ 
      3  MERGE-SORT( $A, 3, 3$ )
          1  if  $3 < 3$ 
      4  MERGE-SORT( $A, 4, 4$ )
          1  if  $4 < 4$ 
      5  MERGE( $A, 3, 3, 4$ )
      MERGE( $A, 1, 2, 4$ )
  
```



- 当一个算法含有对其自身的递归调用时，其运行时间总可以用一个递归方程（或递归式）来表示。该方程通过描述子问题与原问题的关系，来给出总的运行时间。我们可以利用数学工具来解递归式，并给出算法性能的界。
- 分治法给出的时间：

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq c \\ aT(n/b) + D(n) + C(n) & \text{otherwise} \end{cases}$$

- $D(n)$ 是把原问题分解为子问题所花的时间；
- $C(n)$ 是把子问题的解合并为原问题的解所花的时间；
- $T(n)$ 是一个规模为 n 的问题的运行时间。

23

```
MERGE-SORT( $A, p, r$ )  
1  if  $p < r$   
2    Then  $q \leftarrow \lfloor (p+r)/2 \rfloor$   
3      MERGE-SORT( $A, p, q$ )  
4      MERGE-SORT( $A, q+1, r$ )  
5      MERGE( $A, p, q, r$ )
```



- 为简化算法分析，通常假设 n 为2的幂次，使得每次分解产生的子序列长度恰为 $n/2$ 。这一假设并不影响递归式解的增长量级。
- 合并排序 n 个数最坏运行时间：
 - ① 当 $n=1$ 时，合并排序一个元素的时间是个常量；
 - ② 当 $n>1$ 时，运行时间分解如下：
 - 分解：仅仅是计算出子数组的中间位置，需要常量时间， $D(n) = \Theta(1)$ ；
 - 解决：递归地求解两个规模为 $n/2$ 的子问题，时间为 $2T(n/2)$ ；
 - 合并：MERGE过程的运行时间为 $C(n) = \Theta(n)$ 。

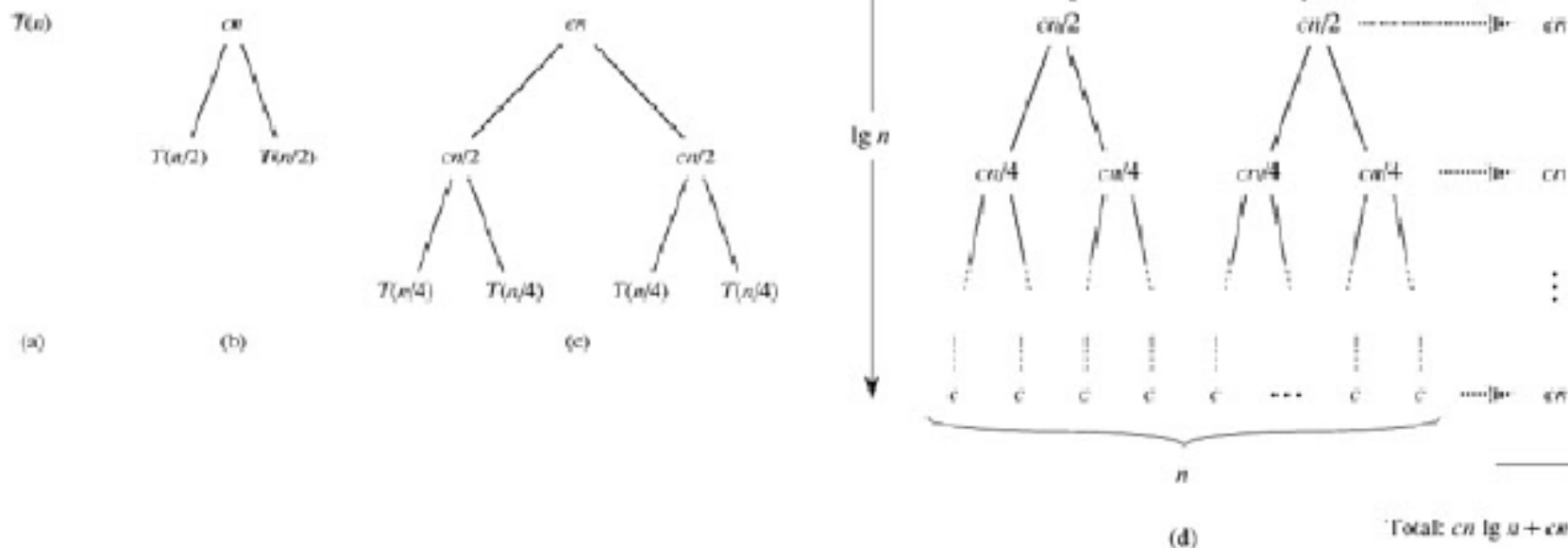
$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

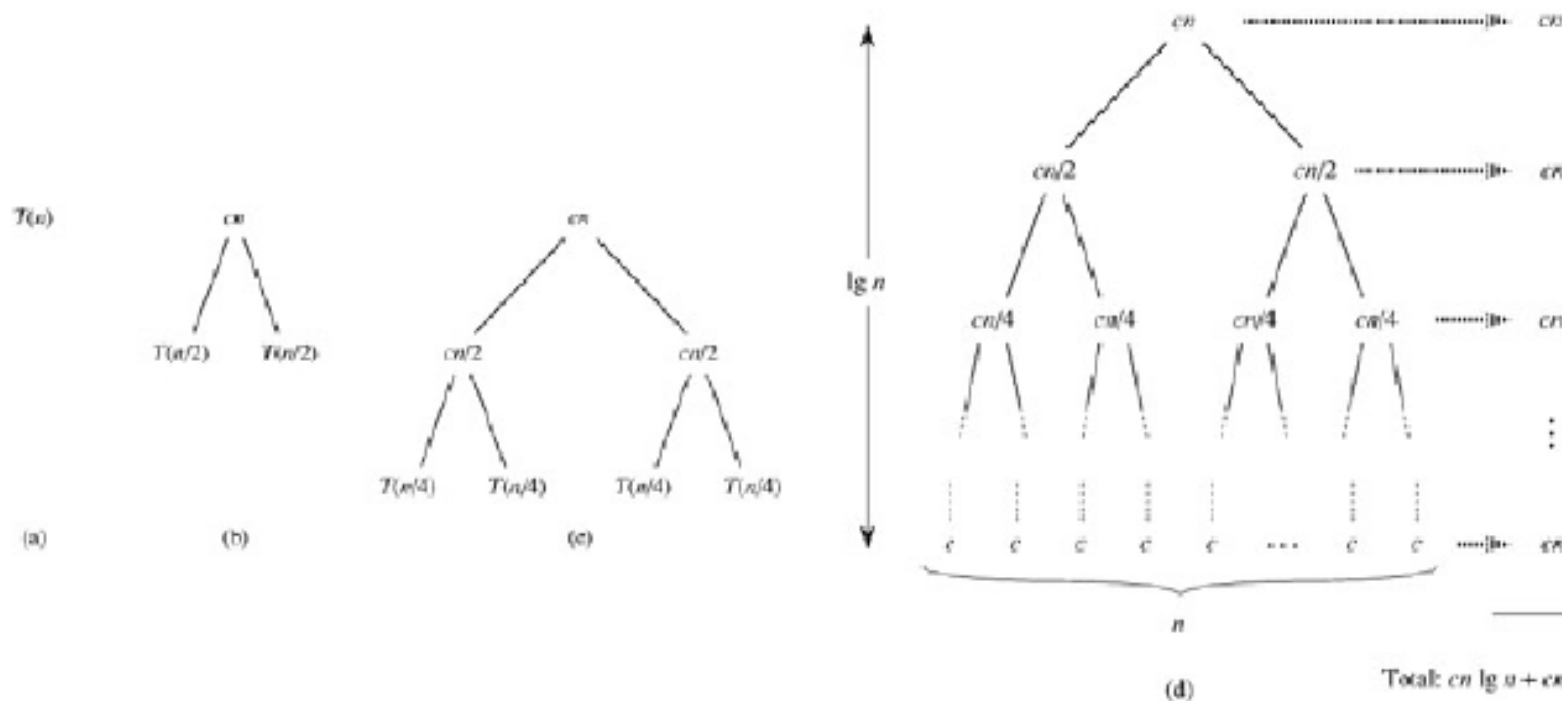


➤ 递归式重写为:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases} \quad \Rightarrow \quad T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + cn & \text{if } n > 1 \end{cases}$$

➤ 递归式求解如下:





➤ 给这棵树的每一层加上代价:

最底层之下的第 i 层有 2^i 个节点, 每一个代价是 $c(n/2^i)$, 第 i 层总代价是 $2^i c(n/2^i) = cn$ 。最底层有 n 个节点, 每一个节点的代价是 c , 该层总代价为 cn ;

➤ 树总共层数是 $\lg n + 1$ 层, 每一层代价都是 cn , 所以总代价为:

$$cn(\lg n + 1) = cn \lg n + cn = \Theta(n \lg n)$$

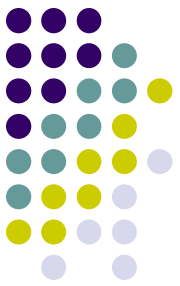
➤ 最坏情况明显比插入排序要好 $\Theta(n^2)$

递归与分治策略



学习要点:

- 理解递归的概念。
- 掌握设计有效算法的分治策略。
- 通过下面的范例学习分治策略设计技巧。
 - (1) 二分搜索技术;
 - (2) 大整数乘法;
 - (3) **Strassen**矩阵乘法;
 - (4) 棋盘覆盖;
 - (5) 合并排序和快速排序;
 - (6) 线性时间选择;
 - (7) 最接近点对问题;
 - (8) 循环赛日程表。



递归的故事

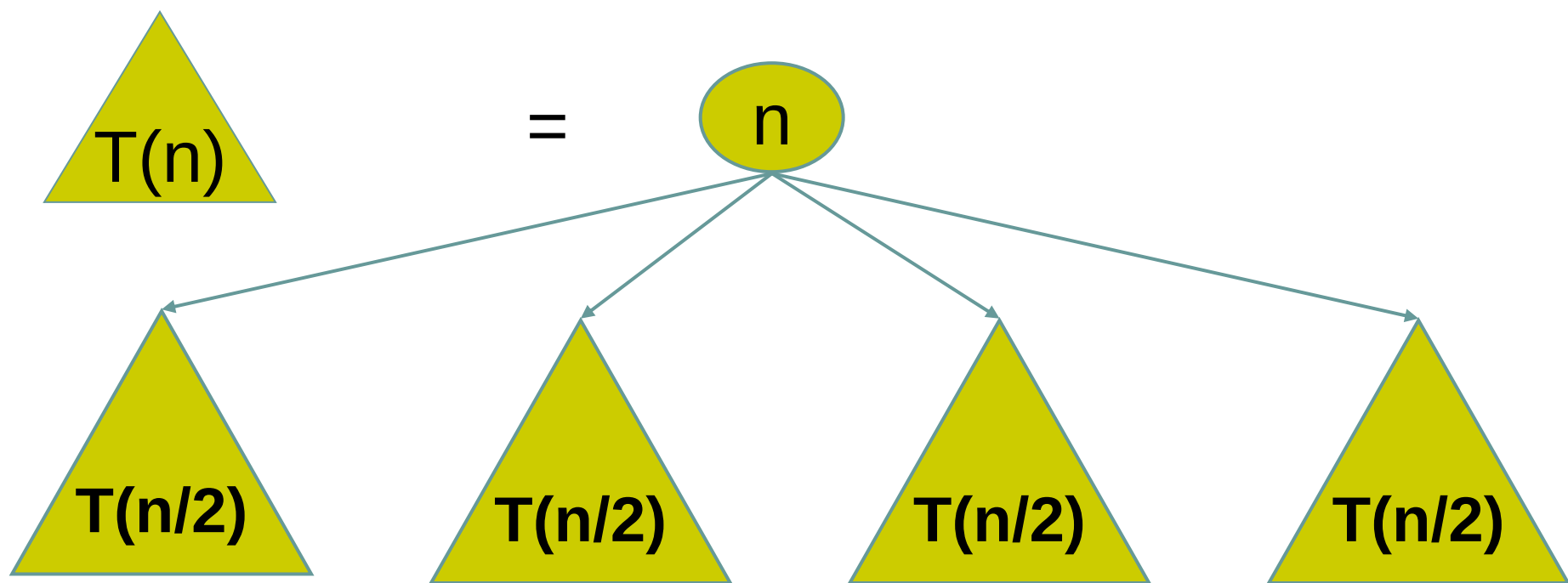
- 从前有座山，山里有座庙，庙里有个老和尚，老和尚对小和尚说：
 - 从前有座山，山里有座庙，庙里有个老和尚，老和尚对小和尚说：
 - 从前有座山，山里有座庙，庙里有个老和尚，老和尚对小和尚说：
 - 有限性：
算法中每条指令的执行次数是有限的
-

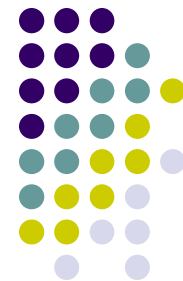




算法总体思想

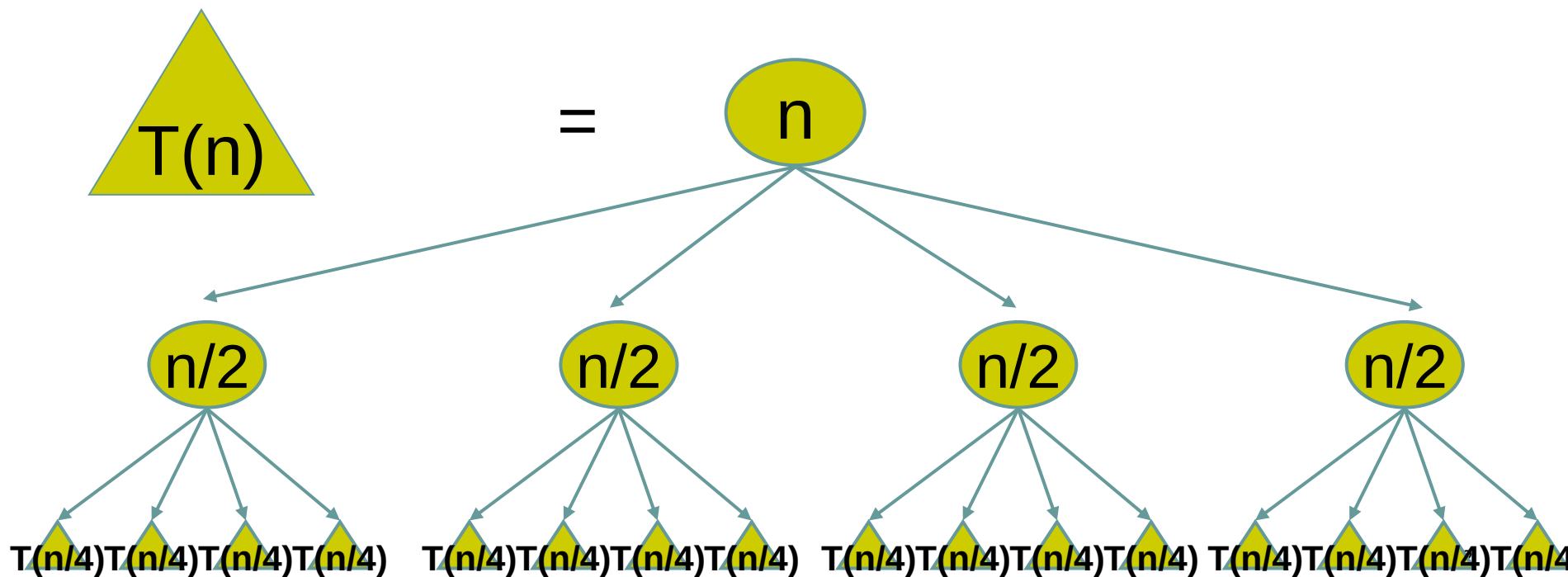
- 对这 k 个子问题分别求解。如果子问题的规模仍然不够小，则再划分为 k 个子问题，如此递归的进行下去，直到问题规模足够小，很容易求出其解为止。





算法总体思想

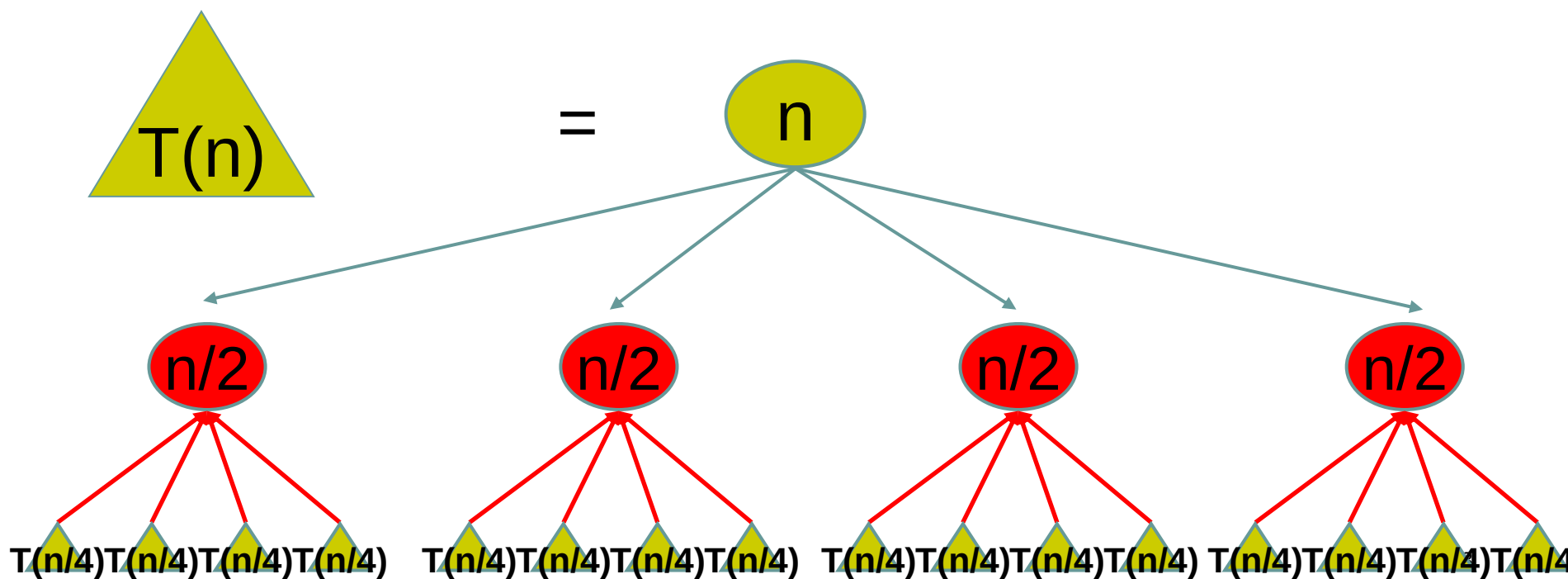
- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。

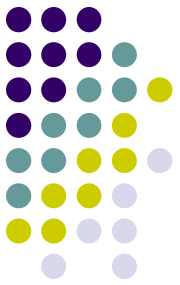




算法总体思想

- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。

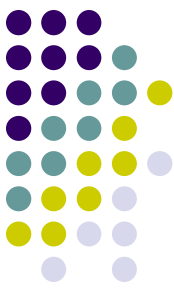




算法总体思想

- 将求出的小规模的问题的解合并为一个更大规模的问题的解，自底向上逐步求出原来问题的解。

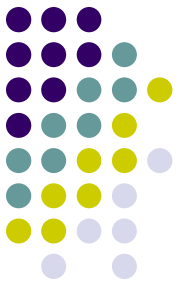
分治法的设计思想是，将一个难以直接解决的大问题，分割成一些规模较小的相同问题，以便各个击破，分而治之。



2.1 递归的概念

- 直接或间接地调用自身的算法称为**递归算法**。用函数自身给出定义的函数称为**递归函数**。
- 由分治法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。在这种情况下，反复应用分治手段，可以使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。这自然导致递归过程的发生。
- 分治与递归像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。

下面来看几个实例。



2.1 递归的概念

例1 阶乘函数

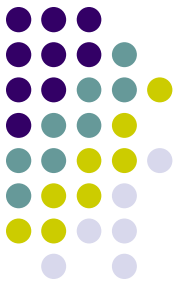
阶乘函数可递归地定义为：

$$n! = \begin{cases} 1 & n = 0 \\ n(n-1)! & n > 0 \end{cases}$$

边界条件

递归方程

边界条件与递归方程是递归函数的二个要素，递归函数只有具备了这两个要素，才能在有限次计算后得出结果。



2.1 递归的概念

例2 Fibonacci数列

无穷数列1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ……，称为Fibonacci数列。它可以递归地定义为：

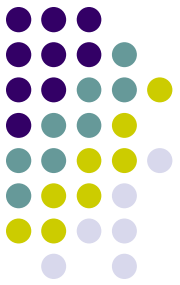
$$F(n) = \begin{cases} 1 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

边界条件

递归方程

第n个Fibonacci数可递归地计算如下：

```
int fibonacci(int n)
{
    if (n <= 1) return 1;
    return fibonacci(n-1)+fibonacci(n-2);
}
```



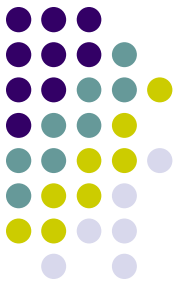
2.1 递归的概念

例3 Ackerman函数

当一个函数及它的一个变量是由函数自身定义时，称这个函数是**双递归函数**。

Ackerman函数 $A(n, m)$ 定义如下：

$$\left\{ \begin{array}{ll} A(1,0) = 2 \\ A(0, m) = 1 & m \geq 0 \\ A(n,0) = n + 2 & n \geq 2 \\ A(n, m) = A(A(n-1, m), m-1) & n, m \geq 1 \end{array} \right.$$



2.1 递归的概念

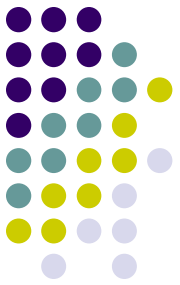
例3 Ackerman函数

前2例中的函数都可以找到相应的非递归方式定义：

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

$$F(n) = \frac{1}{\sqrt{5}} \left(\left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1} \right)$$

本例中的Ackerman函数却无法找到非递归的定义。



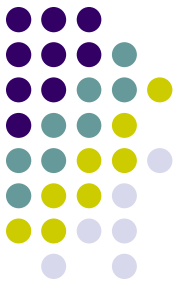
2.1 递归的概念

例3 Ackerman函数

- $A(n, m)$ 的自变量 m 的每一个值都定义了一个单变量函数：
- $M=0$ 时， $A(n, 0) = n + 2$
- $M=1$ 时， $A(n, 1) = A(A(n-1, 1), 0) = A(n-1, 1) + 2$ ，和 $A(1, 1) = 2$
故 $A(n, 1) = 2 * n$
- $M=2$ 时， $A(n, 2) = A(A(n-1, 2), 1) = 2A(n-1, 2)$ ，和
 $A(1, 2) = A(A(0, 2), 1) = A(1, 1) = 2$ ，故 $A(n, 2) = 2^n$ 。

$$\underbrace{2^{2^{\cdot^{\cdot^2}}}}_n$$

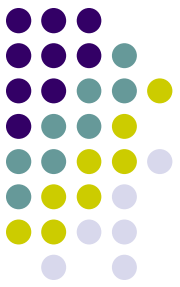
- $M=3$ 时，类似的可以推出
- $M=4$ 时， $A(n, 4)$ 的增长速度非常快，以至于没有适当的数学式子来表示这一函数。



2.1 递归的概念

例3 Ackerman函数

- 定义单变量的Ackerman函数 $A(n)$ 为, $A(n) = A(n, n)$ 。
- 定义其拟逆函数 $\alpha(n)$ 为: $\alpha(n) = \min\{k \mid A(k) \geq n\}$ 。
即 $\alpha(n)$ 是使 $n \leq A(k)$ 成立的最小的 k 值。
- $\alpha(n)$ 在复杂度分析中常遇到。对于通常所见到的正整数 n , 有 $\alpha(n) \leq 4$ 。但在理论上 $\alpha(n)$ 没有上界, 随着 n 的增加, 它以难以想象的慢速度趋向正无穷大。



2.1 递归的概念

例4 排列问题

设计一个递归算法生成 n 个元素 $\{r_1, r_2, \dots, r_n\}$ 的全排列。

设 $R = \{r_1, r_2, \dots, r_n\}$ 是要进行排列的 n 个元素， $R_i = R - \{r_i\}$ 。

集合 X 中元素的全排列记为 $\text{perm}(X)$ 。

$(r_i)\text{perm}(X)$ 表示在全排列 $\text{perm}(X)$ 的每一个排列前加上前缀得到的排列。 R 的全排列可归纳定义如下：

当 $n=1$ 时， $\text{perm}(R) = (r)$ ，其中 r 是集合 R 中唯一的元素；
当 $n>1$ 时， $\text{perm}(R)$ 由 $(r_1)\text{perm}(R_1)$ ， $(r_2)\text{perm}(R_2)$ ， $\dots$ ， $(r_n)\text{perm}(R_n)$ 构成。

生成排列



- 问题是生成 $\{1, 2, \dots, n\}$ 的所有 $n!$ 个排列

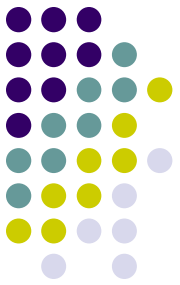
- 想法 1: 固定位置放元素

假设我们能够生成 $n-1$ 个元素的所有排列，我们可以得到如下算法：

生成元素 $\{2, 3, \dots, n\}$ 的所有排列，并且将元素1放到每个排列的开头；

接着，生成元素 $\{1, 3, \dots, n\}$ 的所有排列，并将数字2放到每个排列的开头；

重复这个过程，直到元素 $\{2, 3, \dots, n-1\}$ 的所有排列都产生，并将元素 n 放到每个排列的开头。



GeneratingPerm1()

1 **for** $j \leftarrow 1$ **to** n **do**

2 $P[j] \leftarrow j$

3 Perm1(1)

Perm1(m)

1 **if** $m=n$ **then** output $P[1..n]$

2 **else**

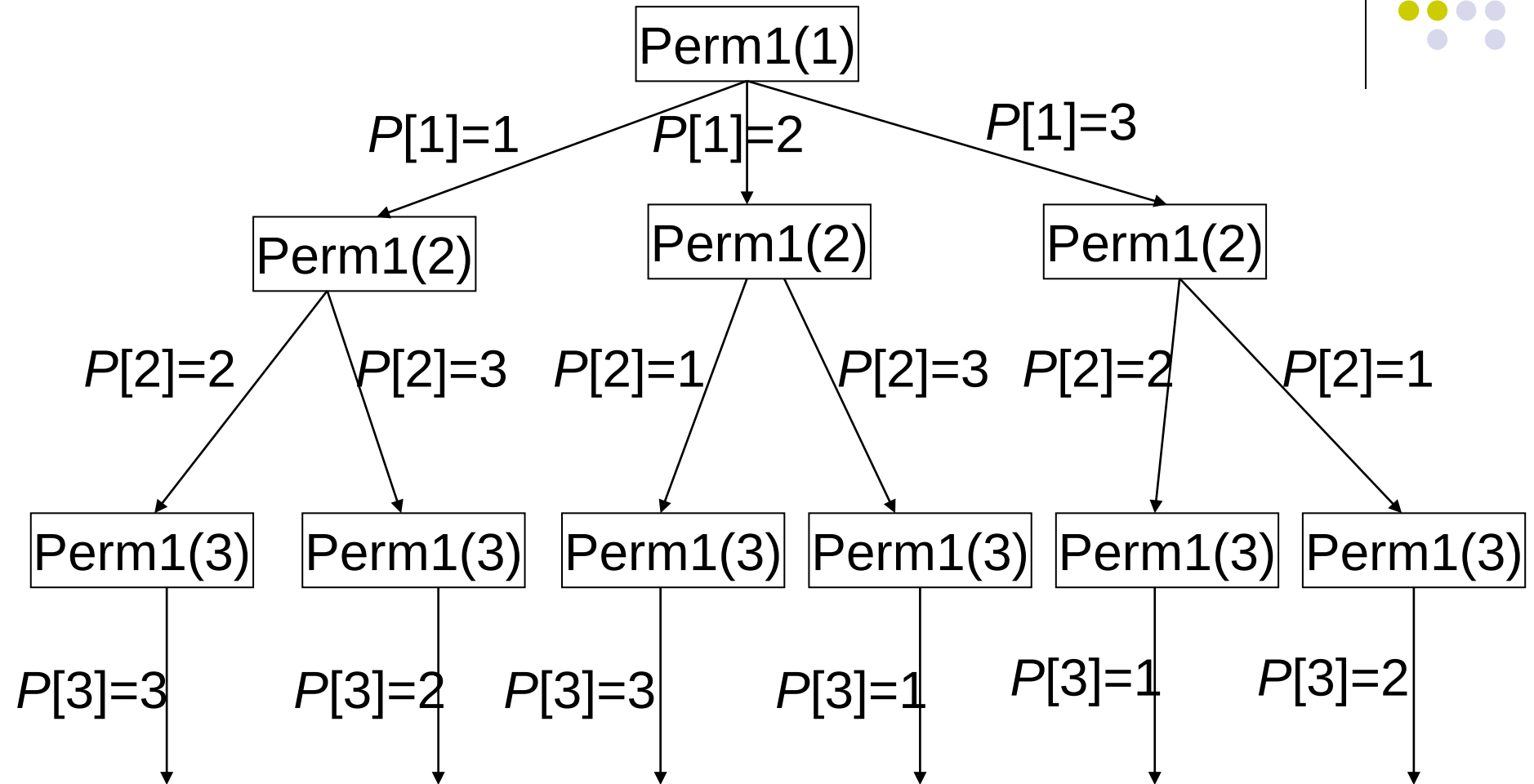
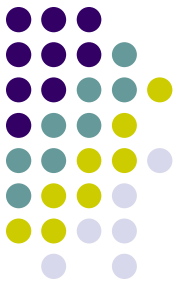
3 **for** $j \leftarrow m$ **to** n **do**

4 $P[j] \leftrightarrow P[m]$

5 Perm1($m+1$)

6 $P[j] \leftrightarrow P[m]$

Illustration for $n=3$.

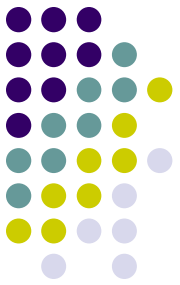


时间复杂度分析

递归方程

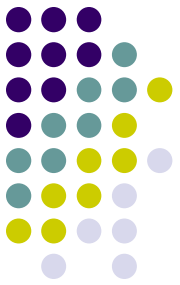


$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ nT(n-1) + n & \text{if } n > 1 \end{cases}$$



想法 2: 固定元素找位置

- 首先，我们把 n 放在的位置 $P[1]$ 上，并且用子数组 $P[2..n]$ 来产生前 $n-1$ 个数的排列；
- 接着，我们将 n 放在 $P[2]$ 上，并且用子数组 $P[1]$ 和 $P[3..n]$ 来产生前 $n-1$ 个数的排列；
- 然后，我们将 n 放在 $P[3]$ 上，并且用子数组 $P[1..2]$ 和 $P[4..n]$ 来产生前 $n-1$ 个数的排列；
- 重复上述过程直到我们将 n 放在 $P[n]$ 上，并且用子数组 $P[1..n]$ 来产生前 $n-1$ 个数的排列。



GeneratingPerm2()

```
1  for  $j \leftarrow 1$  to  $n$  do  
2       $P[j] \leftarrow 0$   
3  Perm2( $n$ )
```

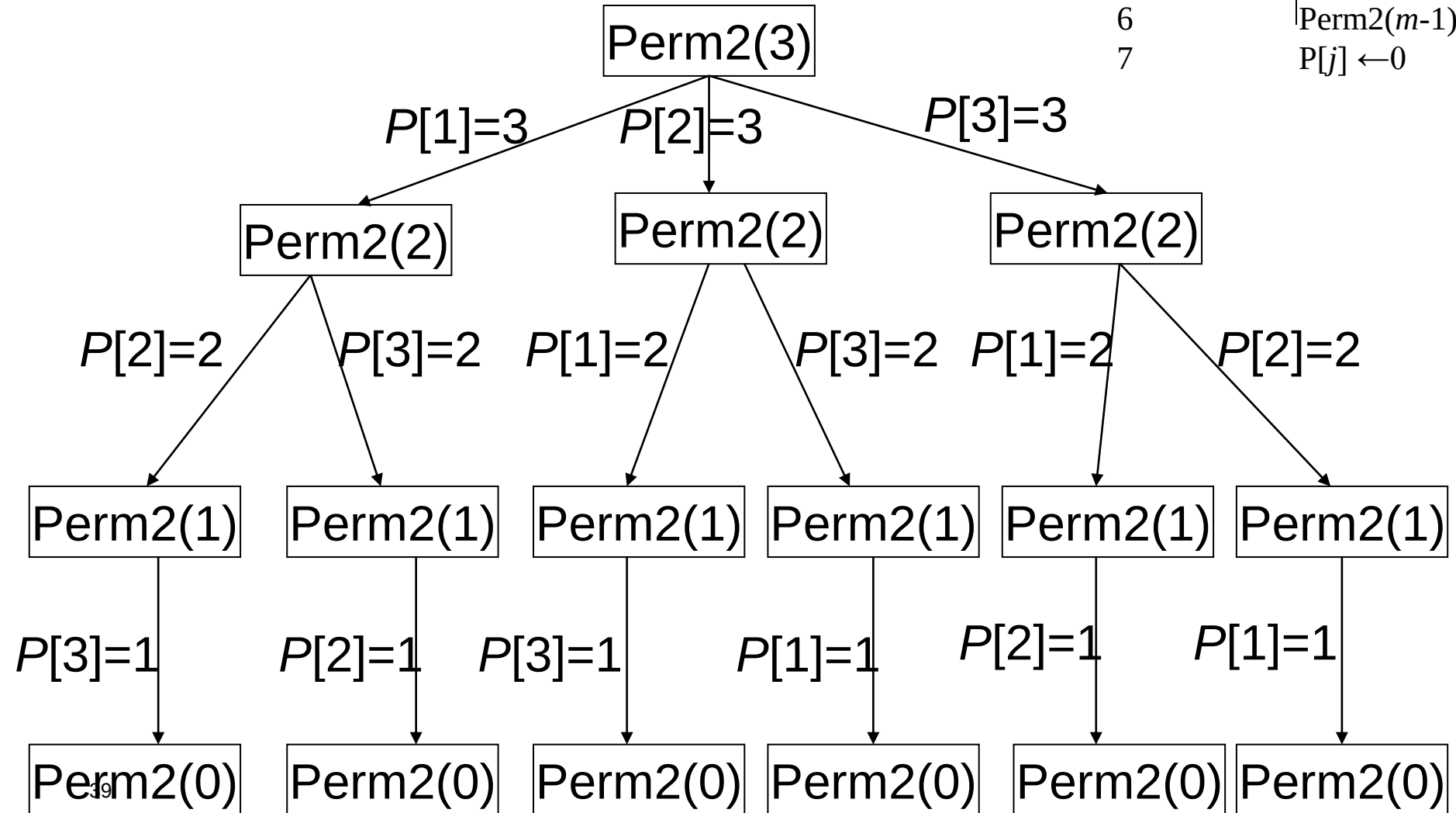
Perm2(m)

```
1  if  $m=0$  then output  $P[1..n]$   
2  else  
3      for  $j \leftarrow 1$  to  $n$  do  
4          if  $P[j]=0$  then  
5               $P[j] \leftarrow m$   
6              Perm2( $m-1$ )  
7               $P[j] \leftarrow 0$ 
```

Illustration for $n=3$

```

Perm2(m)
1  if  $m=0$  then output P
2  else
3      for  $j \leftarrow 1$  to  $n$  do
4          if  $P[j]=0$  then
5               $P[j] \leftarrow m$ 
6              Perm2( $m-1$ )
7               $P[j] \leftarrow 0$ 
    
```

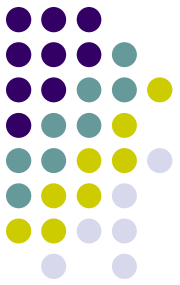


时间复杂度分析

递归方程



$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ nT(n-1) + n & \text{if } n > 1 \end{cases}$$



2.1 递归的概念

例5 整数划分问题

将正整数 n 表示成一系列正整数之和： $n=n_1+n_2+\dots+n_k$ ，

其中 $n_1 \geq n_2 \geq \dots \geq n_k \geq 1$ ， $k \geq 1$ 。

正整数 n 的这种表示称为正整数 n 的划分。求正整数 n 的不同划分个数。

例如正整数6有如下11种不同的划分：

6;

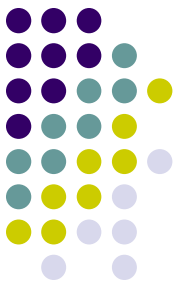
5+1;

4+2, 4+1+1;

3+3, 3+2+1, 3+1+1+1;

2+2+2, 2+2+1+1, 2+1+1+1+1;

1+1+1+1+1+1。



2.1 递归的概念

例5 整数划分问题

前面的几个例子中，问题本身都具有比较明显的递归关系，因而容易用递归函数直接求解。

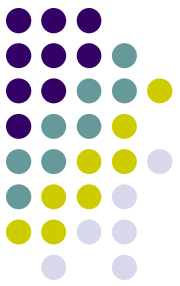
在本例中，如果设 $p(n)$ 为正整数 n 的划分数，则难以找到递归关系，因此考虑增加一个自变量：将最大加数 n_1 不大于 m 的划分个数记作 $q(n, m)$ 。可以建立 $q(n, m)$ 的如下递归关系。

$$(3) \quad q(n, n) = 1 + q(n, n-1);$$

正整数 n 的划分由 $n_1=n$ 的划分和 $n_1 \leq n-1$ 的划分组成。

$$(4) \quad q(n, m) = q(n, m-1) + q(n-m, m), \quad n > m > 1;$$

正整数 n 的最大加数 n_1 不大于 m 的划分由 $n_1=m$ 的划分和 $n_1 \leq n-1$ 的划分组成。



2.1 递归的概念

例5 整数划分问题

前面的几个例子中，问题本身都具有比较明显的递归关系，因而容易用递归函数直接求解。

在本例中，如果设 $p(n)$ 为正整数 n 的划分数，则难以找到递归关系，因此考虑增加一个自变量：将最大加数 n_1 不大于 m 的划分个数记作 $q(n, m)$ 。可以建立 $q(n, m)$ 的如下递归关系。

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

正整数 n 的划分数 $p(n) = q(n, n)$ 。